

INFORME TÉCNICO

CODEFEST AD ASTRA 2026 - Etapa 1

Equipo: Los 100011

José Gabriel Caro, Juan David Huerfano, Ashlee Yin, Mariana Torres

Introducción

El presente documento describe la arquitectura y las principales decisiones de diseño tomadas para el desarrollo de la etapa 1.

El objetivo del sistema es recuperar, desde una base de documentos previamente proporcionada, fragmentos y documentos semánticamente más relevantes. Para ello, se realizó extracción y limpieza de texto, chunking, generación de embeddings mediante modelos encoder y almacenamiento de los vectores en índices FAISS.

La arquitectura fue diseñada teniendo en cuenta las características del corpus, que comprende documentos relacionados con los tres fenómenos definidos por el reto.

Extracción de texto inicial

Cada formato de origen presentó estructuras y problemas de calidad distintos, por lo que se desarrolló un extractor independiente por tipo de archivo, todos escribiendo a un almacén común (fragments.jsonl).

Para los archivos .JSON:

Se identificaron tres estructuras distintas (diccionario de artículo, lista con contenido temático, lista de metadata y lista vacía). Para diccionarios con `body_text` y `body_paragraphs` simultáneos, se aplicó una heurística para detectar redundancia y evitar duplicar contenido o perder información complementaria. Y asimismo, se trató cada estructura con mucha precisión. (Evidencias en `src\extractors\extract_json.py`)

Para los archivos .PDF:

Usando la librería `pymupdf`, se hizo extracción página por página. Se detectan y eliminan líneas repetidas entre páginas de un mismo documento (encabezados/pies de página) mediante

frecuencia relativa. Se normalizan ligaduras Unicode (NFKC) y se colapsan saltos de línea de ajuste visual preservando párrafos reales (`\n\n`).

Para los archivos .PBF:

Se utilizó la librería `mapbox-vector-tile`. Los datos se identificaron usando un código administrativo estable (`slug nivell1-nivel2`) en lugar de las coordenadas de las teselas, porque un mismo municipio puede aparecer en varias teselas y niveles de zoom. También se eliminaron los registros que no tenían datos analíticos.

Para los archivos de imagen:

A través de `tesseract` las imágenes se clasifican automáticamente como informativas (gráficos, tablas) o decorativas (fotografías, retratos) según la densidad de texto alfabético reconocido tras el OCR. Solo las informativas se guardan en `fragments.jsonl`.

Para los archivos .CSV/.XLSX

Mediante el uso de `pandas`, cada fila se trató como unidad de fragmentación, serializada como pares `columna: valor`. Se maneja de forma robusta la inferencia de delimitador y encoding.

Para los archivos .TXT

Mediante expresiones regulares se identifica y descarta el bloque inicial de navegación característico del contenido scrapeado de sitios web, buscando la primera secuencia de tres oraciones de al menos 40 caracteres. El texto restante se utiliza como contenido real del documento.

Estrategia de chunking

La estrategia de fragmentación seleccionada fue híbrida y diferenciada según el tipo de contenido extraído. Para los archivos `pbf`, cada registro de origen correspondiente a un municipio con sus respectivos atributos, se mantuvo como unidad completa, sin realizar una subdivisión adicional. Esto permitió mantener la cohesión semántica de la información geoespacial. Para los archivos `json`, `pdf`, `txt`, `csv` y `xlsx`, se realizó una segmentación para oraciones mediante `NLTK Punkt`, utilizando el modelo de idioma correspondiente de cada fragmento. Posteriormente, las oraciones se acumularon de forma secuencial hasta alcanzar un límite de 500 tokens.

Este límite se estableció teniendo en cuenta que bge-m3 permite trabajar con entradas de hasta 8192 tokens, más se quería conservar una representación semántica más específica de cada fragmento y evitar que una cantidad excesiva de información se concentre en un único vector. De esta manera el límite de entrada del encoder se utilizó como margen para evitar cortes forzados, y no como un tamaño objetivo para los fragmentos.

Además, se mantuvieron las oraciones completas durante el proceso de fragmentación, evitando perder contexto por cortes arbitrarios. En los casos en los que una oración superó por sí sola el límite establecido, esta se conservó completa como un único fragmento. También se implementó un filtro posterior para descartar aquellos fragmentos compuestos principalmente por secuencias numéricas aisladas, correspondientes principalmente a leyendas o ejes de gráficos que podían ser segmentados incorrectamente en documentos pdf.

Este paso fue necesario para obtener fragmentos suficientemente pequeños para ser representados de manera específica por el encoder, pero suficientemente completos para conservar el contexto semántico. Cada fragmento se almacenó junto con la metadata necesaria para mantener la trazabilidad hasta el documento original.

Sin embargo, esta estrategia presenta algunas limitaciones al momento de ejecutarse, principalmente debido al costo computacional asociado al encoder seleccionado, tanto en tiempo como en memoria. Para disminuir este impacto, la codificación se realizó por lotes y utilizando GPU.

Selección del encoder

Se seleccionó BAI/bge-m3 como encoder para generar embeddings para todo el corpus, debido a que permite trabajar con entradas de texto de hasta 8192 tokens, diferenciándolo de otros embeddings y garantizando contar con un margen suficiente para conservar contenido coherente. Además, tomando en cuenta las múltiples instrucciones del reto, se tuvo muy en cuenta la capacidad multilingüe del encoder para poder procesar los idiomas pedidos, los embeddings generados presentan una calidad semántica destacable para mejorar la búsqueda posterior, y cuenta con licencia MIT, cumpliendo con las condiciones establecidas para los modelos utilizados en el reto.

No obstante, se presentó el problema dado que bge-m3 presenta un costo computacional mayor frente a alternativas más ligeras, sin embargo, este se solucionó mediante la ejecución de la

codificación en GPU, utilizando una T4 en Google Colab, debido a las limitaciones de cómputo local.

Construcción del índice FAISS

Para construir el índice vectorial se utilizó IndexFlatIP que realiza una búsqueda exacta mediante producto interno. Antes de insertar los embeddings en el índice, se verificó que sus vectores estuvieran normalizados a norma unitaria y, en caso de no estarlo, se normalizaron explícitamente. De esta manera, el producto interno entre dos vectores corresponde a su similitud coseno.

El índice fue seleccionado con el objetivo de evitar introducir aproximaciones en la búsqueda de los fragmentos más similares, ya que el consumo de memoria y tiempo se mitigó usando codificación en GPU como dijimos anteriormente. Sin embargo, se pudo utilizar métodos de búsqueda aproximada como IVF o HNSW.

La generación de embeddings se realizó por lotes, con una inserción progresiva de los vectores en el índice y checkpoints periódicos. Esto permitió controlar el uso de memoria durante el procesamiento del corpus y conservar el progreso en caso de que se presentaran interrupciones durante la ejecución.

Flujo de Recuperación

El flujo de recuperación implementado sigue el proceso definido en la especificación: la consulta se codifica utilizando el mismo encoder BGE-M3 empleado para construir el índice y se normaliza mediante `normalize_embeddings=True`. Posteriormente, se realiza una búsqueda en FAISS utilizando producto interno, recuperando los 100 fragmentos candidatos más similares. A partir de los identificadores internos retornados por FAISS, se consulta la metadata correspondiente en el almacén externo, obteniendo información como `chunk_id`, `doc_id` y el texto del fragmento.

Para la agregación a nivel documento se utiliza max-pooling: la puntuación asignada a cada documento corresponde a la mayor similitud obtenida por alguno de sus fragmentos dentro de los 100 candidatos recuperados. Esta estrategia evita que los documentos con una mayor cantidad de fragmentos recuperados obtengan una ventaja artificial por acumulación de puntuaciones y

permite priorizar la relevancia del fragmento más relacionado con la consulta. A partir de estas puntuaciones se ordenan los documentos y se seleccionan los tres primeros para la salida final, en correspondencia con la evaluación mediante $F1@3$.

Para los fragmentos incluidos en la salida se aplica un límite de 250 palabras. Cuando un fragmento supera este límite, se divide utilizando fronteras de oración detectadas mediante expresiones regulares, procurando no realizar cortes dentro de una oración. Los sub-fragmentos generados conservan el `chunk_id` y `doc_id` originales para mantener la trazabilidad. Finalmente, se incluyen hasta 10 fragmentos en la salida, manteniendo el orden en que fueron recuperados por FAISS.

En ninguna etapa del módulo de recuperación se emplea un modelo generativo o decoder. La recuperación, el cálculo de similitud, el ranking de documentos y la construcción de la salida se realizan exclusivamente a través de embeddings, FAISS, puntuaciones de similitud y metadata estructurada.

Conclusiones y recomendaciones

El sistema desarrollado cumple al pie de la letra los requisitos solicitados durante la Etapa 1, realizando extracción específica por formato con limpieza y normalización, chunking con completitud lingüística garantizada, codificación multilingüe mediante BGE-M3, indexación exacta con FAISS (IndexFlatIP) y un módulo de recuperación que respeta la restricción de no usar modelos generativos en ninguna etapa posterior a la extracción. Las decisiones de diseño documentadas en este informe buscan maximizar la precisión de recuperación priorizando la coherencia semántica de los fragmentos.

Para futuros proyectos, es recomendable usar más de un encoder para hacer comparación de resultados y buscar la mayor precisión posible. También sería conveniente experimentar con diferentes estrategias de chunking y tamaños de fragmento, ya que estos pueden afectar directamente la relevancia de los resultados.

Aclaraciones sobre el repositorio

En el repositorio se omite la carpeta data/raw, como se ve en el .gitignore, esto debido a que ahí está el corpus recibido y son en conjunto demasiado grandes para subirlos al repositorio.

Asimismo, si desean acceder a fragments.jsonl, chunks.jsonl, metadata.jsonl y index.faiss, es necesario ejecutar git lfs install, seguido de git lfs pull. Esto debido a que github tiene un límite de 100 MB por archivo individual y un límite recomendado de menos de 1 GB a 5 GB por repositorio completo, los cuales estos archivos superan, por lo que resulta imposible subirlo al repositorio sin esta condición.

Para acceder al repositorio completo, con todos los archivos usados además de los entregables haga clic [aquí](#).